3uwbyaohj

March 3, 2025

1 Singer vs Rapper Classification

2 Import Library

[ ]:
```
# Import library yang dibutuhkan
import os  # Untuk manajemen file dan direktori
import numpy as np  # Untuk operasi numerik
import pandas as pd  # Untuk manipulasi data
import tensorflow as tf  # Untuk membangun dan melatih model deep learning
print(f'TensorFlow version: {tf.__version__}')  # Cek versi TensorFlow

# Library untuk membangun model neural network
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Activation, Flatten
from tensorflow.keras.optimizers import Adam
# Import ModelCheckpoint here
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.losses import BinaryCrossentropy
from sklearn.metrics import classification_report  # Untuk evaluasi model

# Library untuk pemrosesan data dan evaluasi
from sklearn import metrics
from sklearn.model_selection import train_test_split

# Library untuk pemrosesan audio
import librosa
import librosa.display
import soundfile as sf

# Library untuk visualisasi
import seaborn as sns
import matplotlib.pyplot as plt
import IPython.display as ipd  # Untuk menampilkan audio dalam notebook

# Alat bantu tambahan
from itertools import cycle  # Untuk mengatur siklus warna dalam grafik
```

```
# Mengatur tema visualisasi seaborn
sns.set_theme(style='white', palette=None)
color_pal = plt.rcParams["axes.prop_cycle"].by_key()['color']
color_cycle = cycle(plt.rcParams["axes.prop_cycle"].by_key()['color'])
```

```
TensorFlow version: 2.18.0
```

3 Mengimpor dan Memuat File Audio

Menghubungkan Google Colab dengan Google Drive

[ ]:
```
# Menghubungkan Google Colab dengan Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

Mengimpor File Audio dari Google Drive

[ ]:
```
import os

# Tentukan path ke folder audio di dalam Google Drive
singer_path = "/content/drive/MyDrive/audio_classification/Artists/Singer"
rapper_path = "/content/drive/MyDrive/audio_classification/Artists/Rapper"

# Mengimpor file audio dari folder Singer
s = []
for root, dirs, files in os.walk(singer_path, topdown=False):
    for name in files:
        if name != '.DS_Store':  # Hindari file sistem yang tidak relevan
            abs_path = os.path.join(root, name)
            s.append(abs_path)

# Mengimpor file audio dari folder Rapper
r = []
for root, dirs, files in os.walk(rapper_path, topdown=False):
    for name in files:
        if name != '.DS_Store':
            abs_path = os.path.join(root, name)
            r.append(abs_path)

s  # Menampilkan daftar file audio di folder Singer
```

[ ]:
```
['/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-
Bieber-2-Much-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-All-
Around-Me-(TrendyBeatz.com).mp3',
```

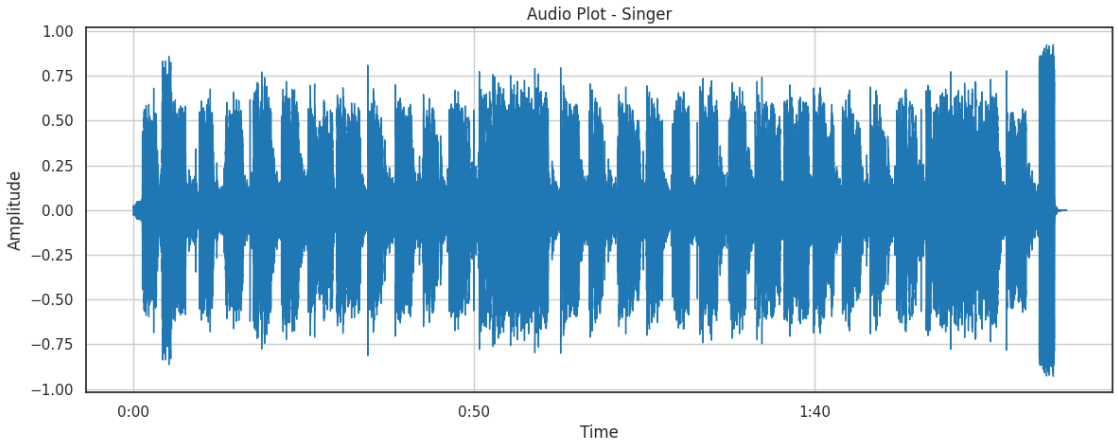
```
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Afraid-To-Say-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Anyone-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-As-I-
Am-Ft-Khalid-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Available-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Deserve-You-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-At-
Least-For-Now-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Ft-
Burna-Boy-Loved-By-You-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Freedom-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Die-
For-You-Ft-Dominic-Fike-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Ft-
DJ-Khaled-21-Savage-Let-It-Go-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Ft-
Post-Malone-Clever-Forever-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Ft-
The-Kid-LAROI-Unstable-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Habitual-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Lonely-Benny-Blanco-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Peaches-Ft-Daniel-Caesar-Giveon-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Love-
You-Different-Ft-Beam-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-Off-
My-Face-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Second-Emotion-feat-Travis-Scott-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Somebody-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Running-Over-feat-Lil-Dicky-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-ft-
Quavo-Intentions-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin-Bieber-
Thats-What-Love-Is-(TrendyBeatz.com).mp3',
 '/content/drive/MyDrive/audio_classification/Artists/Singer/Justin_Bieber_Stuck
_with_U_(thinkNews.com.ng).mp3']
```

Visualisasi Gelombang Suara (Waveform)

Singer

[ ]:
```
import librosa.display

# Memuat file audio dari daftar Singer
x, sr = librosa.load(s[1])

# Menampilkan bentuk gelombang audio
plt.figure(figsize=(14, 5))
plt.grid()
librosa.display.waveshow(x, sr=sr, color=color_pal[0])
plt.title("Audio Plot - Singer")
plt.ylabel("Amplitude")

# Menampilkan audio player agar bisa didengarkan
display(ipd.Audio(x, rate=sr))
```

```
<IPython.lib.display.Audio object>
```

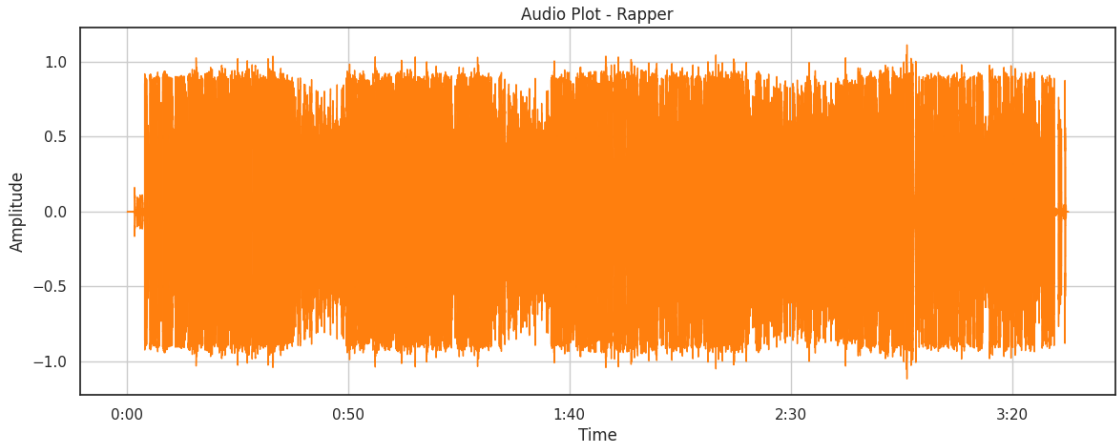


Rapper

[ ]:
```
# Memuat file audio dari daftar Rapper
x, sr = librosa.load(r[1])

# Menampilkan bentuk gelombang audio
plt.figure(figsize=(14, 5))
plt.grid()
librosa.display.waveshow(x, sr=sr, color=color_pal[1])
plt.title("Audio Plot - Rapper")
plt.ylabel("Amplitude")
```

```
# Menampilkan audio player agar bisa didengarkan
display(ipd.Audio(x, rate=sr))
```

```
<IPython.lib.display.Audio object>
```



4 Ekstraksi Vokal dari Lagu Menggunakan Spleeter

[ ]:
```
###  Fungsi untuk Mengekstrak Vokal dari Banyak File Audio Menggunakan Spleeter
# Fungsi ini menjalankan Spleeter pada setiap file audio dalam daftar␣
 ↪(input_list)
# dan menyimpan hasil ekstraksi vokalnya di folder output_dir.

# def extract_vocals(input_list, output_dir):
#     for i in input_list:
#         command = 'spleeter separate ' + i + ' -o ' + output_dir  # Perintah␣
 ↪untuk menjalankan Spleeter
#         os.system(command)  # Menjalankan perintah di terminal

# # Membuat folder untuk menyimpan vokal dari file Singer
# output_dir = '/singer-vs-rapper/Vocals/Singer'
# os.makedirs(output_dir, exist_ok=True)
# extract_vocals(s, output_dir)  # Menjalankan ekstraksi vokal untuk Singer

# # Membuat folder untuk menyimpan vokal dari file Rapper
# output_dir = '/singer-vs-rapper/Vocals/Rapper'
# os.makedirs(output_dir, exist_ok=True)
# extract_vocals(r, output_dir)  # Menjalankan ekstraksi vokal untuk Rapper
```

5 Mengimpor Vokal untuk Eksplorasi Data (EDA)

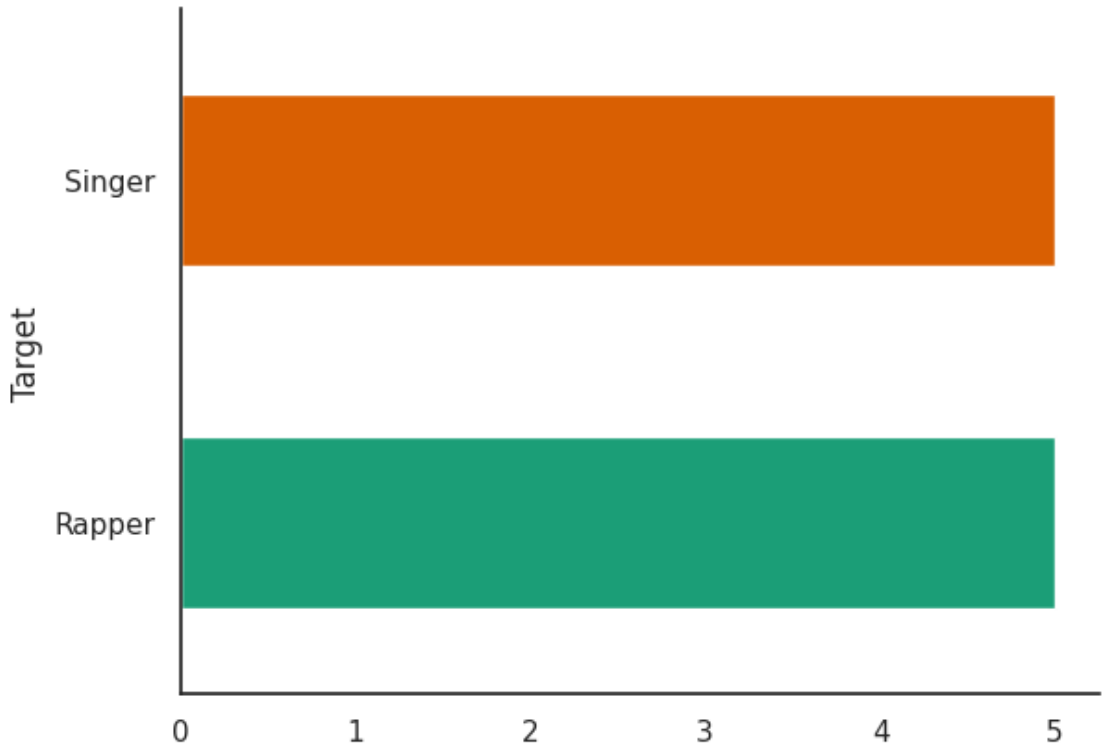
[ ]:
```
vocals = []  # Menyimpan path file vokal
target = []  # Menyimpan label (Singer/Rapper)

# Memuat file vokal dari folder Singer
for root, dirs, files in os.walk("/content/drive/MyDrive/audio_classification/
 ↪Vocals/Singer", topdown=False):
    for name in files:
        if name != '.DS_Store' and name != 'accompaniment.wav':  # Hindari file␣
 ↪sistem & instrumen
            abs_path = os.path.join(root, name)
            vocals.append(abs_path)
            target.append('Singer')

# Memuat file vokal dari folder Rapper
for root, dirs, files in os.walk("/content/drive/MyDrive/audio_classification/
 ↪Vocals/Rapper", topdown=False):
    for name in files:
        if name != '.DS_Store' and name != 'accompaniment.wav':
            abs_path = os.path.join(root, name)
            vocals.append(abs_path)
            target.append('Rapper')

# Membuat DataFrame untuk menyimpan informasi file vokal
df = pd.DataFrame({'Vocals': vocals, 'Target': target})
df  # Menampilkan DataFrame
```

[ ]:

	Vocals	Target
0	/content/drive/MyDrive/audio_classification/Vo…	Singer
1	/content/drive/MyDrive/audio_classification/Vo…	Singer
2	/content/drive/MyDrive/audio_classification/Vo…	Singer
3	/content/drive/MyDrive/audio_classification/Vo…	Singer
4	/content/drive/MyDrive/audio_classification/Vo…	Singer
5	/content/drive/MyDrive/audio_classification/Vo…	Rapper
6	/content/drive/MyDrive/audio_classification/Vo…	Rapper
7	/content/drive/MyDrive/audio_classification/Vo…	Rapper
8	/content/drive/MyDrive/audio_classification/Vo…	Rapper
9	/content/drive/MyDrive/audio_classification/Vo…	Rapper

visualisasi distribusi data berdasarkan kategori “Target” (Singer atau Rapper)

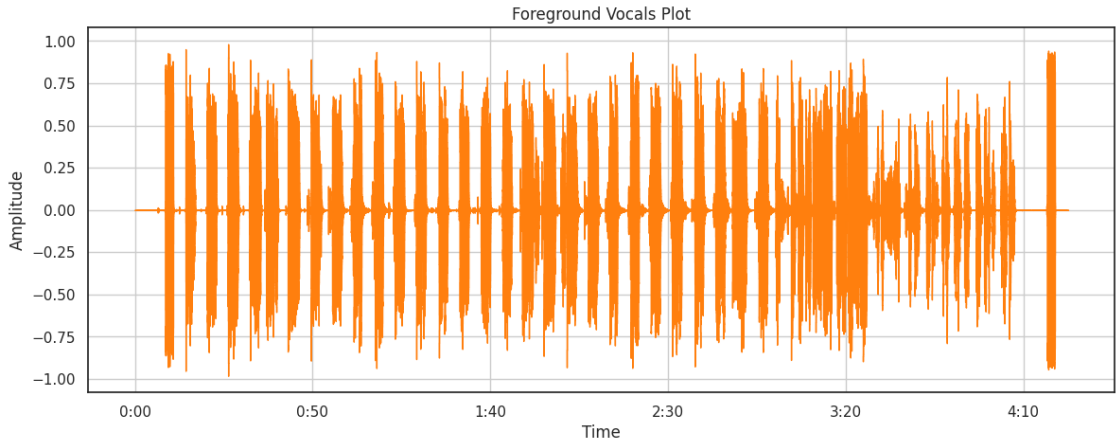
[ ]:
```
from matplotlib import pyplot as plt
import seaborn as sns
df.groupby('Target').size().plot(kind='barh', color=sns.palettes.
 ↪mpl_palette('Dark2'))
plt.gca().spines[['top', 'right',]].set_visible(False)
```



6 Memvisualisasikan Gelombang Suara Vokal

[ ]:
```
plt.figure(figsize=(14, 5))  # Atur ukuran plot
plt.grid()  # Tambahkan grid agar lebih mudah dibaca

# Memuat file vokal pertama dalam DataFrame df
x, sr = librosa.load(df.Vocals[0])

# Menampilkan bentuk gelombang suara
librosa.display.waveshow(x, sr=sr, color=color_pal[1])
plt.title("Foreground Vocals Plot")  # Judul plot
plt.ylabel("Amplitude")  # Label sumbu Y
```

[ ]:
```
Text(109.74999999999999, 0.5, 'Amplitude')
```



[ ]:
```
# Memutar audio
ipd.Audio(x, rate=sr)
```

[ ]:
```
<IPython.lib.display.Audio object>
```

7 Menghapus bagian audio yang sunyi (silence)

[ ]:
```
nonMuteSections = librosa.effects.split(x, top_db=20)
nonMuteSections
```

[ ]:
```
array([[ 186368,  238592],
       [ 313856,  356352],
       [ 357888,  372224],
       [ 444928,  505856],
       [ 574464,  643072],
       [ 708608,  762880],
       [ 763392,  781312],
       [ 804864,  806400],
       [ 813568,  835584],
       [ 836096,  862720],
       [ 864256,  868864],
       [ 869376,  883712],
       [ 884736,  887296],
       [ 947712,  990208],
       [ 991232, 1020928],
       [1075200, 1079808],
       [1090560, 1102848],
       [1103360, 1155584],
       [1207296, 1213952],
       [1222656, 1252864],
```

```
[1253376, 1286144],
[1339904, 1346560],
[1352192, 1406976],
[1472000, 1479168],
[1484288, 1502720],
[1503744, 1517056],
[1518080, 1539584],
[1607680, 1610752],
[1611264, 1671680],
[1743360, 1805312],
[1884672, 1939968],
[2013696, 2030080],
[2030592, 2071040],
[2145792, 2172928],
[2174464, 2205696],
[2278912, 2311680],
[2312192, 2338304],
[2398208, 2422784],
[2423808, 2456576],
[2457088, 2470912],
[2472448, 2475008],
[2481664, 2505216],
[2521600, 2523648],
[2530304, 2577920],
[2578944, 2608640],
[2639872, 2670592],
[2678272, 2690560],
[2691072, 2743808],
[2796032, 2798080],
[2810368, 2873856],
[2927616, 2933248],
[2939392, 2994688],
[3059712, 3065344],
[3072000, 3090432],
[3091456, 3104768],
[3105792, 3127296],
[3191808, 3261440],
[3324416, 3330560],
[3331072, 3394560],
[3471872, 3529728],
[3589632, 3593216],
[3600896, 3617280],
[3618304, 3662848],
[3705344, 3794432],
[3853824, 3858432],
[3866624, 3926016],
[3974144, 3999744],
```

```
[4002816, 4021248],
[4046336, 4051968],
[4069888, 4097024],
[4113408, 4135424],
[4135936, 4147200],
[4158976, 4192768],
[4204544, 4234752],
[4235264, 4267520],
[4269568, 4300288],
[4300800, 4308480],
[4312576, 4324352],
[4350464, 4370432],
[4370944, 4381696],
[4382208, 4412416],
[4420096, 4481536],
[4483072, 4492800],
[4498432, 4541440],
[4581888, 4585984],
[4600832, 4614656],
[4627968, 4643328],
[4647424, 4723712],
[4729856, 4731904],
[4734464, 4738560],
[4743168, 4744192],
[4779520, 4786176],
[4787200, 4788736],
[4794880, 4805120],
[4807680, 4815360],
[4835328, 4849664],
[4850688, 4864000],
[4865536, 4866048],
[4909056, 4930560],
[4932096, 4934144],
[4943872, 4951040],
[4951552, 4953600],
[4956160, 4960256],
[4967424, 4978688],
[5005312, 5011968],
[5012480, 5022720],
[5029376, 5051904],
[5083136, 5097984],
[5101568, 5104128],
[5104640, 5111296],
[5113344, 5117440],
[5119488, 5120000],
[5125120, 5126144],
[5145088, 5174784],
```

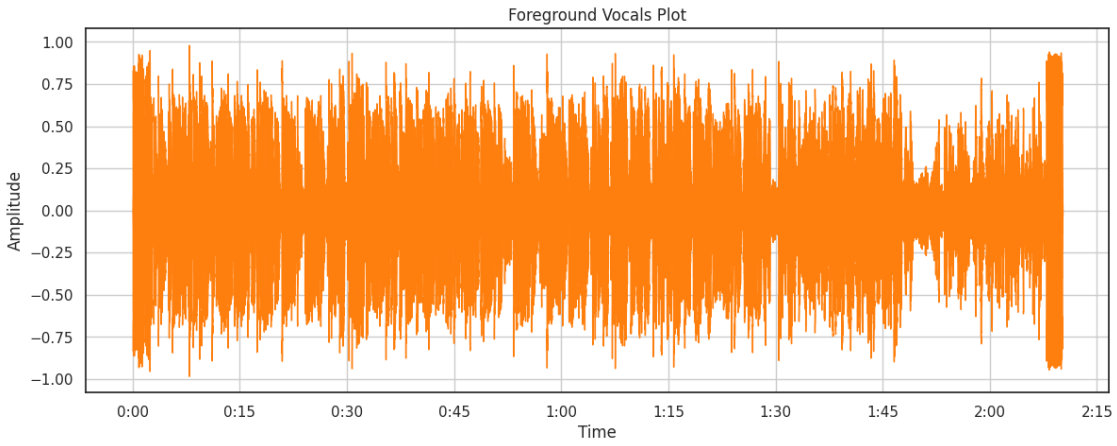
```
[5217280, 5240832],
[5241344, 5246976],
[5267968, 5281280],
[5283328, 5299712],
[5300736, 5303808],
[5318144, 5325824],
[5327360, 5329920],
[5371392, 5379584],
[5380608, 5381632],
[5384192, 5386240],
[5387264, 5392896],
[5393920, 5408256],
[5423616, 5428224],
[5429760, 5442048],
[5445632, 5449728],
[5451776, 5453824],
[5657600, 5709824]])
```

[ ]:
```
wav = np.concatenate([x[start:end] for start, end in nonMuteSections])
```

menampilkan bentuk gelombang (waveform) dari audio

[ ]:
```
plt.figure(figsize=(14, 5))  # Membuat figure dengan ukuran 14x5 inci
plt.grid()  # Menambahkan grid untuk memudahkan analisis visual

librosa.display.waveshow(wav, sr=sr, color=color_pal[1])
# Menampilkan bentuk gelombang audio menggunakan warna dari color_pal

plt.title("Foreground Vocals Plot")  # Judul plot
a = plt.ylabel("Amplitude")  # Label sumbu Y menunjukkan amplitudo audio

ipd.Audio(wav, rate=sr)  # Memutar audio setelah pemrosesan
```

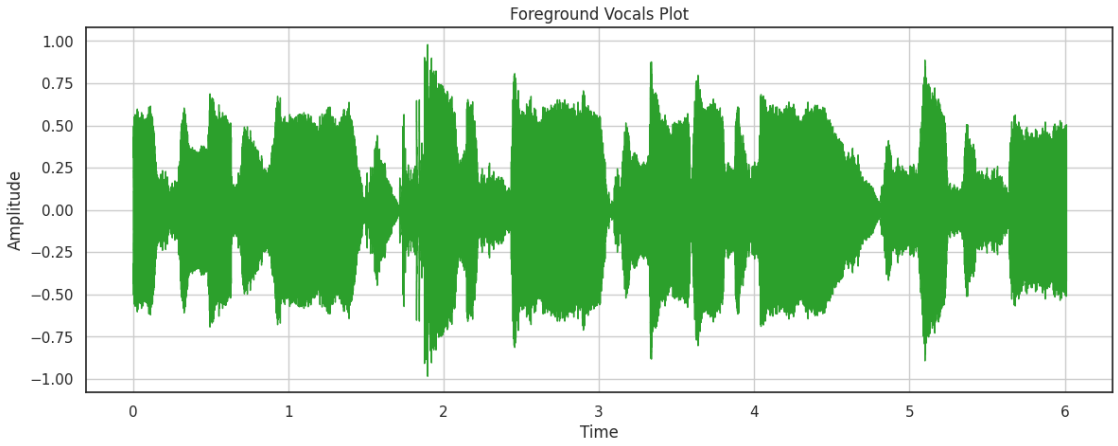
[ ]:
```
<IPython.lib.display.Audio object>
```



8 Ekstraksi Sampel Audio (~6 detik)

[ ]:
```
a = 132500  # Menentukan jumlah sampel untuk sekitar 6 detik
interval_1 = wav[a:a*2]  # Mengambil bagian audio dari posisi a sampai 2*a␣
↪(sekitar 6 detik)

plt.figure(figsize=(14, 5))  # Membuat figure dengan ukuran 14x5 inci
plt.grid()  # Menampilkan grid

librosa.display.waveshow(interval_1, sr=sr, color=color_pal[2])
# Menampilkan bentuk gelombang (waveform) dari audio yang dipotong

plt.title("Foreground Vocals Plot")  # Menambahkan judul
a = plt.ylabel("Amplitude")  # Menampilkan label sumbu Y

ipd.Audio(interval_1, rate=sr)  # Memutar audio hasil ekstraksi
```

[ ]:
```
<IPython.lib.display.Audio object>
```



9 Menghapus Silence & Menyimpan Sampel 5 Detik

[ ]:
```
dir_path = '/content/drive/MyDrive/audio_classification/Samples'
os.makedirs(dir_path, exist_ok=True)
```

[ ]:
```
# menentukan panjang interval (132500 sampel  6 detik jika sr=22050 Hz)
interval_length = 132500
```

```
# Fungsi untuk menghapus bagian senyap & membagi audio menjadi potongan kecil
def foreground_process(input_list, final_directory, interval_length=132500,␣
 ↪number_of_intervals=6):
    count = 1
    os.makedirs(final_directory, exist_ok=True)

    for i in input_list:
        x, sr = librosa.load(i)  # Memuat file audio

        # Menghapus bagian senyap dari audio
        nonMuteSections = librosa.effects.split(x, top_db=20)
        wav = np.concatenate([x[start:end] for start, end in nonMuteSections])

        # Membagi audio menjadi beberapa potongan kecil (5-6 detik per potongan)
        for j in range(number_of_intervals):
            start = (j+1) * interval_length  # Indented this line
            end = (j+2) * interval_length  # Indented this line

            # Cek apakah masih ada cukup audio untuk diambil
            if start < len(wav):  # Indented this line and the following block
                interval = wav[start:end]
                sf.write(f"{final_directory}/{count}.wav", interval, sr,␣
 ↪'PCM_24')
                count += 1

# Menyimpan sampel audio penyanyi
final_directory = '/content/drive/MyDrive/audio_classification/Samples/Singer'
foreground_process(df.Vocals[df.Target == 'Singer'], final_directory)

# Menyimpan sampel audio rapper
final_directory = '/content/drive/MyDrive/audio_classification/Samples/Rapper'
foreground_process(df.Vocals[df.Target == 'Rapper'], final_directory)
```

10 Membuat Dataframe Sampel Audio untuk Klasifikasi

[ ]:
```
# Mengumpulkan path file audio dari folder "Samples" dan memberi label

import os
import pandas as pd

vocals = []
target = []

# Loop untuk mengumpulkan file dari folder Singer dan Rapper
for category in ["Singer", "Rapper"]:
```

```
    folder_path = f"/content/drive/MyDrive/audio_classification/Samples/
 ↪{category}"

    for root, dirs, files in os.walk(folder_path, topdown=False):
        for name in files:
            if name not in ['.DS_Store', 'accompaniment.wav']:
                abs_path = os.path.join(root, name)
                vocals.append(abs_path)
                target.append(category)

# Membuat dataframe
df_samples = pd.DataFrame({'Vocals': vocals, 'Target': target})
df_samples
```

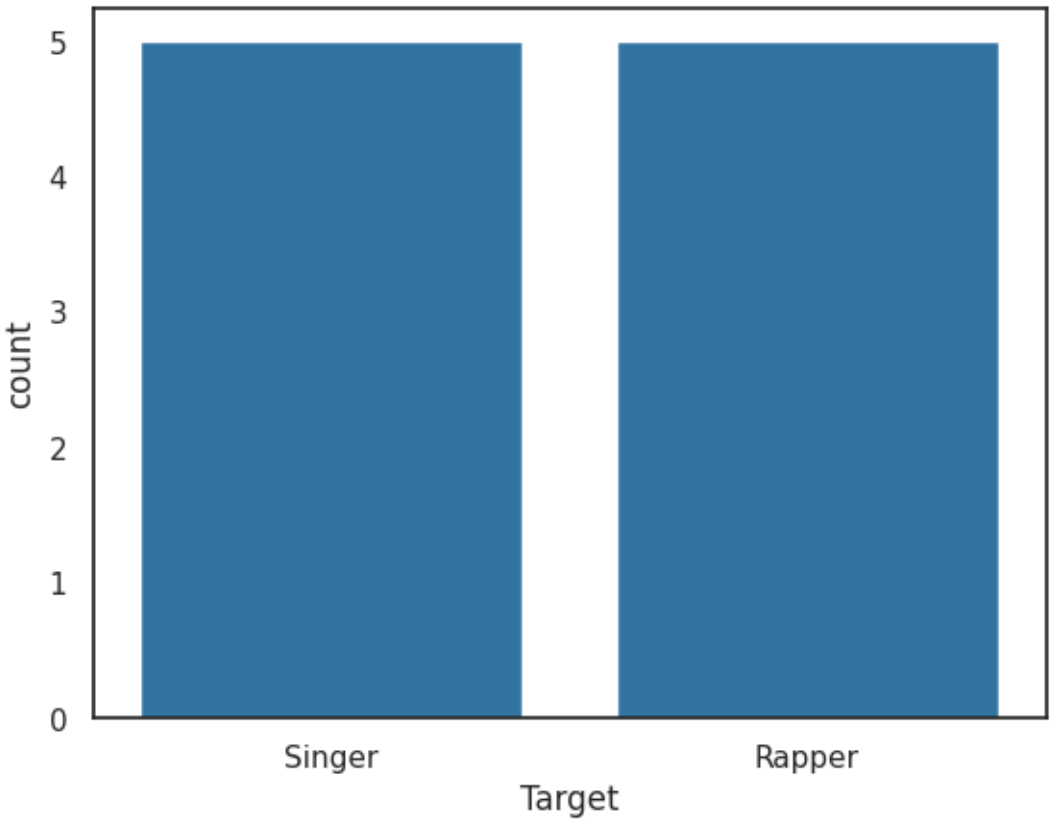
[ ]:

	Vocals	Target
0	/content/drive/MyDrive/audio_classification/Sa…	Singer
1	/content/drive/MyDrive/audio_classification/Sa…	Singer
2	/content/drive/MyDrive/audio_classification/Sa…	Singer
3	/content/drive/MyDrive/audio_classification/Sa…	Singer
4	/content/drive/MyDrive/audio_classification/Sa…	Singer
..	…	…
295	/content/drive/MyDrive/audio_classification/Sa…	Rapper
296	/content/drive/MyDrive/audio_classification/Sa…	Rapper
297	/content/drive/MyDrive/audio_classification/Sa…	Rapper
298	/content/drive/MyDrive/audio_classification/Sa…	Rapper
299	/content/drive/MyDrive/audio_classification/Sa…	Rapper

[300 rows x 2 columns]

mengecek keseimbangan data

[ ]:
```
sns.countplot(data=df, x="Target")  # Menggunakan x untuk orientasi vertikal
plt.show()
```



11 Mengekstrak fitur MFCC (Mel Frequency Cepstral Coefficients)

```
[ ]: mfcc = librosa.feature.mfcc(y=wav, sr=sr)
     mfcc
```

```
[ ]: array([[-418.61523  , -339.92804  , -276.30194  , …, -281.39957  ,
             -323.67987  , -392.2824   ],
            [  87.8971   ,  151.81143  ,  182.835    , …,  150.28885  ,
              136.54895  ,   98.83924  ],
            [  56.93334  ,   76.74265  ,   85.129295 , …,   24.444248 ,
               18.163853 ,   32.42601  ],
            …,
            [  -5.011809 ,    1.9260026,    6.7809153, …,   -8.35857  ,
               0.6319104,    7.828662 ],
            [  -5.0314107,   -7.8823357,  -10.044443 , …,   -6.6296864,
               1.7613649,   11.4788885],
            [  -4.735046 ,   -5.3933115,    3.0787134, …,   -5.778616 ,
              -1.2796319,    2.3926091]], dtype=float32)
```

[ ]:
```
print(mfcc.shape)
```

```
(20, 5608)
```

[ ]:
```
def feature_extractor(file):
    audio, sr = librosa.load(file)  # Membaca file audio
    mfccs_features = librosa.feature.mfcc(y=audio, sr=sr)  # Mengekstrak MFCC
    mfccs_scaled_features = np.mean(mfccs_features.T, axis=0)  # Merata-ratakan␣
 ↪MFCC

    return mfccs_scaled_features  # Mengembalikan hasil
```

[ ]:
```
def batch_extractor(input_list):
    extracted_features = []  # Menyimpan hasil ekstraksi fitur
    for i in input_list:  # Looping setiap file audio
        extracted_features.append(feature_extractor(i))  # Ekstrak MFCC dan␣
 ↪simpan

    return extracted_features  # Mengembalikan daftar fitur

# Ekstrak fitur dari semua file dalam df_samples
extracted_features = batch_extractor(df_samples.Vocals)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
<ipython-input-22-afe89bd9888d> in <cell line: 0>()
      7
      8 # Ekstrak fitur dari semua file dalam df_samples
----> 9 extracted_features = batch_extractor(df_samples.Vocals)

<ipython-input-22-afe89bd9888d> in batch_extractor(input_list)
      2     extracted_features = []  # Menyimpan hasil ekstraksi fitur
      3     for i in input_list:  # Looping setiap file audio
----> 4         extracted_features.append(feature_extractor(i))  # Ekstrak MFCC␣
 ↪dan simpan
      5
      6     return extracted_features  # Mengembalikan daftar fitur

<ipython-input-21-952eb5918a40> in feature_extractor(file)
      1 def feature_extractor(file):
      2     audio, sr = librosa.load(file)  # Membaca file audio
----> 3     mfccs_features = librosa.feature.mfcc(y=audio, sr=sr)  # Mengekstra␣
 ↪MFCC
      4     mfccs_scaled_features = np.mean(mfccs_features.T, axis=0)  #␣
 ↪Merata-ratakan MFCC
      5
```

```
/usr/local/lib/python3.11/dist-packages/librosa/feature/spectral.py in mfcc(y,␣
 ↪sr, S, n_mfcc, dct_type, norm, lifter, **kwargs)
   1987     if S is None:
   1988         # multichannel behavior may be different due to relative noise␣
 ↪floor differences between channels
-> 1989         S = power_to_db(melspectrogram(y=y, sr=sr, **kwargs))
   1990
   1991     M: np.ndarray = scipy.fftpack.dct(S, axis=-2, type=dct_type,␣
 ↪norm=norm)[

/usr/local/lib/python3.11/dist-packages/librosa/feature/spectral.py in␣
 ↪melspectrogram(y, sr, S, n_fft, hop_length, win_length, window, center,␣
 ↪pad_mode, power, **kwargs)
   2128     >>> ax.set(title='Mel-frequency spectrogram')
   2129     """
-> 2130     S, n_fft = _spectrogram(
   2131         y=y,
   2132         S=S,

/usr/local/lib/python3.11/dist-packages/librosa/core/spectrum.py in␣
 ↪_spectrogram(y, S, n_fft, hop_length, power, win_length, window, center,␣
 ↪pad_mode)
   2943         S = (
   2944             np.abs(
-> 2945                 stft(
   2946                     y,
   2947                     n_fft=n_fft,

/usr/local/lib/python3.11/dist-packages/librosa/core/spectrum.py in stft(y,␣
 ↪n_fft, hop_length, win_length, window, center, dtype, pad_mode, out)
    373         off_end = y_frames_post.shape[-1]
    374         if off_end > 0:
--> 375             stft_matrix[…, -off_end:] = fft.rfft(fft_window *␣
 ↪y_frames_post, axis=-2)
    376     else:
    377         off_start = 0

/usr/local/lib/python3.11/dist-packages/numpy/fft/_pocketfft.py in rfft(a, n,␣
 ↪axis, norm)
    407         n = a.shape[axis]
    408     inv_norm = _get_forward_norm(n, norm)
--> 409     output = _raw_fft(a, n, axis, True, True, inv_norm)
    410     return output
    411

/usr/local/lib/python3.11/dist-packages/numpy/fft/_pocketfft.py in _raw_fft(a,␣
 ↪n, axis, is_real, is_forward, inv_norm)
```

```
     71     else:
     72         a = swapaxes(a, axis, -1)
---> 73         r = pfi.execute(a, is_real, is_forward, fct)
     74         r = swapaxes(r, axis, -1)
     75     return r

KeyboardInterrupt:
```

[ ]:
```
df_samples['features'] = extracted_features
df_samples
```

[ ]:
```
# Mengacak urutan dataset agar distribusi lebih merata
df_samples = df_samples.sample(frac=1).reset_index(drop=True)
df_samples
```

[ ]:
```
# Menampilkan label unik dalam dataset
print(df_samples['Target'].unique())
```

[ ]:
```
df_samples['Target'].replace({'Singer': 1, 'Rapper': 0}, inplace=True)

X = np.array(df_samples['features'].tolist())
y = np.array(df_samples['Target'].tolist())
y
```

[ ]:
```
print(X.shape)
print(y.shape)
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.20)
print(X_train.shape)
print(y_train.shape)
```

12 Model Building

[ ]:
```
model = Sequential()

#First layer stack
model.add(Dense(100, input_shape=(20,)))
model.add(Activation('relu'))
model.add(Dropout(0.5))


#Second layer stack
model.add(Dense(200))
model.add(Activation('relu'))
model.add(Dropout(0.5))
```

```
#Third layer stack
model.add(Dense(200))
model.add(Activation('relu'))
model.add(Dropout(0.5))


#Final Layer
model.add(Dense(1))
model.add(Activation("sigmoid"))

model.summary()
```

[ ]:
```
# Save the best model
saved_callbacks = ModelCheckpoint('./content/saved_models/bestmodel.h5',
                                  save_weights_only=False,
                                  monitor='loss',
                                  save_best_only=True)

model.compile(optimizer = Adam(),
              loss = BinaryCrossentropy(),
               metrics = ['accuracy',])

history = model.fit(X_train, y_train, epochs=100, batch_size=32,
                    callbacks=[saved_callbacks]
    )
```

13 Evaluasi Model

[ ]:
```
plt.subplot(2 ,1, 1)
a = plt.plot(history.history['loss'])
plt.ylabel('loss')
plt.xlabel('epoch')


plt.subplot(2, 1 ,2)
b = plt.plot(history.history['accuracy'],color="orange")
plt.ylabel('accuracy')
plt.xlabel('epoch')

plt.tight_layout()
plt.show()
```

[ ]:
```
y_preds = model.predict(X_test)
y_preds = (y_preds>0.5).flatten()
y_preds = y_preds.astype('int')
```

```
y_preds
```

[ ]:
```
print(classification_report(y_preds, y_test))
```

[ ]:
```
# Fungsi untuk memuat file audio dan mengekstrak fitur MFCC
def load_and_extract_features(file_path):
    if os.path.exists(file_path):
        print("File ditemukan:", file_path)
        audio, sr = librosa.load(file_path, sr=16000)
        mfccs_features = librosa.feature.mfcc(y=audio, sr=sr)
        mfccs_scaled_features = np.mean(mfccs_features.T, axis=0)
        return mfccs_scaled_features
    else:
        print("File tidak ditemukan!")
        return None

# Path ke file audio yang akan diuji
file_path = os.path.abspath("/content/drive/MyDrive/audio_classification/
 ↪Samples/Rapper/1.wav")

# Memuat dan mengekstrak fitur dari file audio
features = load_and_extract_features(file_path)

if features is not None:
    # Mengubah fitur menjadi array 2D untuk prediksi
    features = np.array([features])

    # Melakukan prediksi menggunakan model yang sudah dilatih
    prediction = model.predict(features)
    prediction = (prediction > 0.5).astype('int')

    # Menampilkan hasil prediksi
    if prediction == 1:
        print("Prediksi: Singer")
    else:
        print("Prediksi: Rapper")
```

```
File ditemukan: /content/drive/MyDrive/audio_classification/Samples/Rapper/1.wav
1/1             0s 107ms/step
Prediksi: Rapper
```